



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)

Re-Accredited by NAAC with 'A++' Grade

॥ वसुधैव कुटुम्बकम् ॥

Name of Student: Aryan Singh Kasera

PRN: 25070122052

Subject: Programming with Java

Sem: III AY: 2026-27

Assignment No: 2

TITLE : Develop a Java application demonstrating the use of default, parameterized, and copy constructors for object initialization and data management.

Assignment 2: Methods and Static

Members Problem Statement

Develop a Java application demonstrating the use of default, parameterized, and copy constructors for object initialization and data management.

Learning Objectives

To understand and implement default, parameterized, and copy constructors for object initialization.

Theory

Constructors are special member functions that are automatically invoked when an object is created. Java supports default constructors provided by the compiler, parameterized constructors for initializing objects with user-defined values, and copy constructors (implemented manually) for creating duplicate objects. Constructors improve code readability and simplify object initialization while supporting object-oriented programming principles.

Create a Student class using default and parameterized constructors to initialize student name and roll number.

Student.java    Share 

```
1 public class Student {  
2     String name;  
3     long prn;  
4  
5     // Default Constructor  
6     public Student() {  
7         name = "Not Assigned";  
8         prn = 0L;  
9     }  
10  
11    // Parameterized Constructor  
12    public Student(String n, long p) {  
13        name = n;  
14        prn = p;  
15    }  
16  
17    // Display Method  
18    public void display() {  
19        System.out.println("Name: " + name + ", PRN: " + prn);  
20    }  
21  
22    // Main Method  
23    public static void main(String[] args) {  
24        Student s1 = new Student();  
25        Student s2 = new Student("Aryan Singh Kasera", 12);  
26  
27        s1.display();  
28        s2.display();  
29    }  
30 }
```

Output

Name: Not Assigned, PRN: 0
Name: Aryan Singh Kasera, PRN: 12

=== Code Execution Successful ===

2. Develop a Mobile Phone Inventory System using different constructors to initialize mobile details and create duplicate object records.

MobilePhone.java



Run

```
1 public class MobilePhone {
2     String model;
3     int price;
4
5     // Default Constructor
6     public MobilePhone() {
7         model = "Unknown";
8         price = 0;
9     }
10
11    // Parameterized Constructor
12    public MobilePhone(String m, int p) {
13        model = m;
14        price = p;
15    }
16
17    // Copy Constructor
18    public MobilePhone(MobilePhone mobile) {
19        model = mobile.model;
20        price = mobile.price;
21    }
22
23    // Display Method
24    public void show() {
25        System.out.println("Model: " + model + ", Price: $" + price);
26    }
27
28    // Main Method
29    public static void main(String[] args) {
30        MobilePhone m1 = new MobilePhone("Iphone XS", 900);
31
32        MobilePhone m2 = new MobilePhone(m1);
33
34        m1.show();
35        m2.show();
36    }
37 }
```

Output

Model: Iphone XS, Price: \$900

Model: Iphone XS, Price: \$900

=== Code Execution Successful ===

Conclusion

Hence, the implementation of various constructor types in Java was successfully demonstrated to handle object initialization and data management effectively. By utilizing default constructors for baseline setup, parameterized constructors for assigning specific user-defined values, and copy constructors for creating accurate duplicate records, the exercises illustrated flexible and efficient object creation. This approach not only improves overall code readability and simplifies the initialization process, but it also strongly reinforces core object-oriented programming principles by ensuring data consistency and reusability across multiple instances.